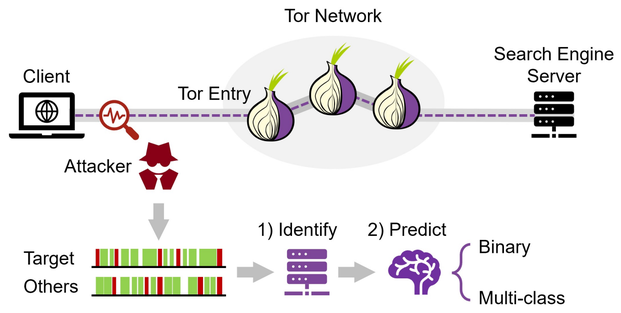
EWHA WOMANS UNIVERSITY

Securing Search Privacy on Tor: Deep Keyword Fingerprinting and BurstGuard Defense

Chai Won Hwang*, Hae Seung Jeon*, Goun Kim, Ji Woo Hong, Ho Sung Kang, Se Eun Oh
Ewha Womans University, Seoul, South Korea

I. ABSTRACT

Anonymous networks like Tor provide user anonymity by employing a 3-hop node routing system, where each node is aware of only its predecessor and successor. However, Keyword Fingerprinting (KF) attacks pose a threat to user privacy by inferring user-typed keywords through the analysis of traffic metadata, such as traffic volume and timing information.



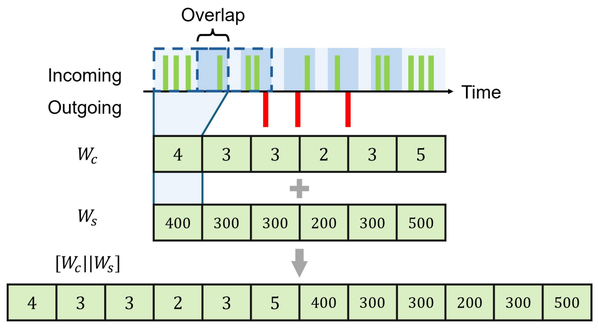
Despite the significant privacy risks associated with KF attacks, current research predominantly utilizes traditional machine learning (ML) techniques. This approach persists even though deep learning (DL) models have demonstrated improvements in analyzing Tor traffic, particularly in Website Fingerprinting (WF). To bridge this gap, we introduce a novel DL-based approach, Deep Keyword Fingerprinting, which utilizes a large dataset and extended WF features to significantly enhance existing fingerprinting models. Additionally, we explore a novel defense mechanism designed to disrupt keyword trace patterns, thereby bolstering search privacy on Tor.

Key Contributions

We extended the Traffic Aggregation Matrix (TAM) and developed a new KF feature that effectively captures keyword traffic patterns.
Moreover, we developed a state-of-the-art KF model that significantly outperforms competing models, with a 41% higher accuracy than Tik-Tok, 48% higher than DF, and 55% higher than k-FP in the DuckDuckGo dataset.
Lastly, we introduce a new padding-based KF defense that outperforms FRONT and WTF-PAD by 19% and 7%, with 66% and 24% reduced bandwidth overhead.

II. METHODS

Winlap Feature



We divided the trace into windows, which is a short fixed-time segment. In each incoming packet window, the packet count and the sum of packet sizes are extracted and concatenated into one feature vector. We divided a 60-second keyword trace into 2,000 subtraces making each window overlap the next by 50% to capture a more detailed traffic pattern. These adjustments led to a 6%p increase in accuracy.

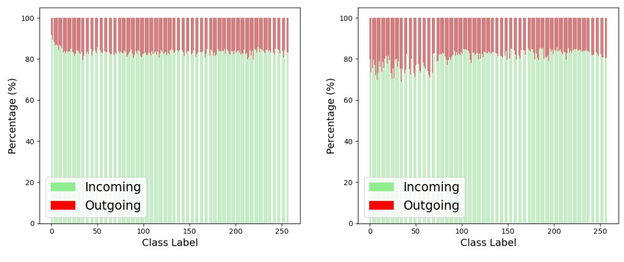
Algorithm 1 Winlap Feature Generation Algorithm

Input: A trace, T_i, overlap ratio, o, window count, w_n, total duration, $d(T_i)$
Output: A feature vector, F_i
$W_c^i \leftarrow \{\}$ // Initialize empty count sum vector
$W_s^i \leftarrow \{\}$ // Initialize empty size sum vector
$w \leftarrow \frac{d(T_i)}{w_n}$ // Fixed-time intervals
$l \leftarrow \left(\frac{d(T_i)-w}{w\cdot(1-o)}\right)+2$ // Total window count
$j \leftarrow 0$ // Window index in F_i
for p_i in T_i **do**
  Extract (t_i, d_i, s_i) from p_i
  $j \leftarrow \min(\left\lfloor \frac{t_i}{w\cdot(1-o)} \cdot l \right\rfloor, l-1)$

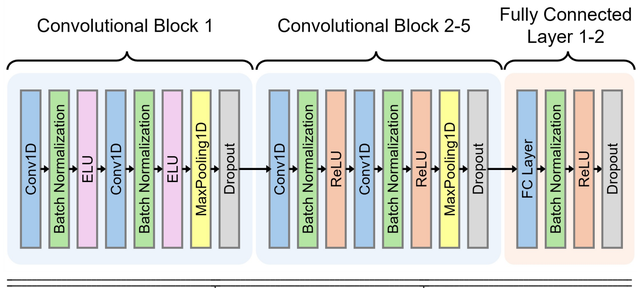
  // Only for incoming packets,
  if $d_i < 0$ **then**
    $W_{c,j}^i \leftarrow W_{c,j}^i + 1$ // Packet count sum
    $W_{s,j}^i \leftarrow W_{s,j}^i + s_i$ // Packet size sum

    // Only for overlapping packets,
    if $t_i < (w\cdot(1-o)\cdot j + w\cdot o)$ **then**
      $W_{c,j-1}^i \leftarrow W_{c,j-1}^i + 1$ // Packet count sum
      $W_{s,j-1}^i \leftarrow W_{s,j-1}^i + s_i$ // Packet size sum
    end if
  end if
end for
return $W_c^i || W_s^i$

Since incoming packets are dominant in network traffic and create distinct patterns that distinguish keywords, we developed a feature that focuses on only incoming packets. This approach was also applied when designing a defense mechanism.



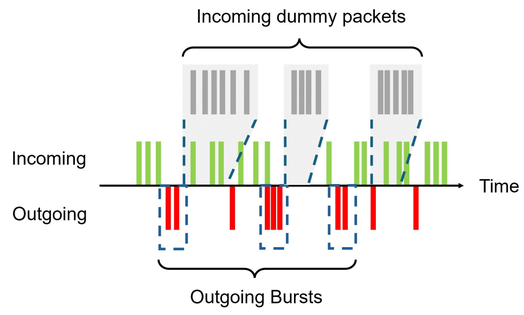
DKF Model



Hyperparameters	Search Boundary	Chosen Parameter
Dropout Rate	[0.1, 0.2, 0.3]	0.2
Batch Size	[32, 64, 128]	32
Block Count	[4, 5, 6]	5
Activation Functions	ReLU, Leaky-ReLU, ELU, Swish	ELU, ReLU
Regularizations	L2, None	None

Our state-of-the-art KF model, called Deep Keyword Fingerprinting (DKF), was extended from Deep Fingerprinting (DF). We carefully tuned the DF, resulting in a slightly deeper architecture than that of DF, comprising five convolutional blocks. Moreover, we tuned hyperparameters like dropout rate, batch size, activation function, and regulations. These changes in the model and new feature design were made to adapt to concept drift in keyword traces.

Burst Guard



The new padding-based defense, called BurstGuard, focuses on interrupting traffic patterns in short intervals. When outgoing bursts, which means two consecutive outgoing packets are detected, BurstGuard adds random incoming packets to make frequent small incoming burst patterns. This effectively hides the original incoming bursts and confuses the classifier.

Algorithm 2 BurstGuard Defense Algorithm

Input: A trace, T_i
Output: A simulated trace, T_i'
$T_i' \leftarrow \{\}$ // Initial empty list
$H \leftarrow$ Initial empty IPD histogram
$t_p \leftarrow 0$ // Timestamp of previous packet
$s_d \leftarrow 1480$ // Size of dummy packet
for p_i **in** T_i **do**
  extract (t_i, d_i, s_i) from p_i
  $t_p \leftarrow t_i$
  update H with $(t_i - t_p)$
  $T_i' \leftarrow T_i' \cup p_i$
  pick n randomly from $[1, 9)$
  if d_i **is** true **and** $p_i \in$ burst **then**
    $b_i' \leftarrow \{\}$ // Dummy burst
    for j **in** range(n) **do**
      $p_{i,j}' \leftarrow ()$ // Dummy packet
      pick δ from the bottom 20% of H
      $p_{i,j}' \leftarrow (t_i + \delta, -1, s_d)$
      $b_i' \leftarrow b_i' \cup p_{i,j}'$
    end for
    $T_i' \leftarrow T_i' \cup b_i'$
  end if
end for

III. Experiments and Results

Data Collection

Visit search engine (Bing, DuckDuckGo) → Search keyword & Collect traffic → Filter CAPTCHA → Extract metadatas → Send data to server

Bing Dataset. For the monitored dataset, each of the 273 monitored keywords was queried 1,200 times across 50 batches in a round-robin manner. After removing faulty traces, we achieved 1,000 trace instances for 258 monitored keywords.
For the unmonitored dataset, we collected one instance per keyword and, after filtering out abnormal traces, achieved a total of 93,443 unmonitored keyword traces.

Duckduckgo Dataset. We collected only the monitored dataset for DuckDuckGo. Similarly to Bing, each of the 273 monitored keywords was queried 1,400 times, distributed across 50 batches in a round-robin manner. As some classes lost samples due to filtering, we reorganized the dataset, limiting it to 1,000 instances per class to ensure balance.

Open World Experiments

TABLE I: DKF attack accuracy in the closed-world scenarios.

# of classes	DKF (%)		Tik-tok (%)		DF (%)		k-FP (%)		Wfin (%)		svmResp (%)		CUMUL (%)	
	Bing	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*
60 classes	**84.65**	**81.60**	56.69	47.55	40.00	36.01	37.79	33.20	13.26	08.22	07.95	06.08	05.87	04.02
200 classes	**63.36**	**72.00**	46.79	31.76	37.29	24.02	30.66	16.00	06.98	02.38	04.05	02.26	03.36	01.56
258 classes	**59.19**	**69.27**	45.56	28.26	34.62	21.64	29.22	14.30	06.78	01.93	03.68	01.85	03.18	01.15

For the comparisons to state-of-the-art WF and KF models, we evaluated these models using the Bing and DuckDuckGo datasets. When distinguishing 258 DuckDuckGo keyword traces, DKF improved accuracy by 41% over Tik-Tok and 48% over DF.

Closed World Experiments

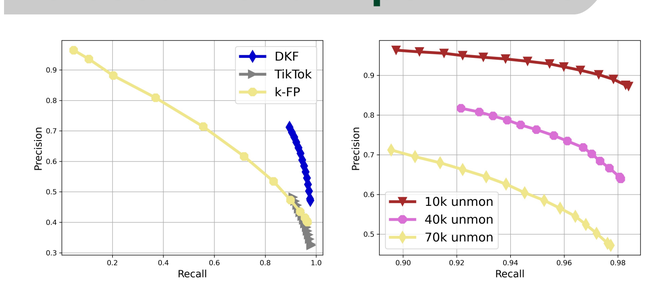


Fig. 4: Comparison of DKF performance with Tik-Tok and k-FP in the open-world scenarios.

We constructed an open-world dataset consisting of 100 monitored keywords, each represented by 1,000 instances, and varied the size of the unmonitored keyword sets from 10,000 to 70,000. We analyzed the performance by plotting precision-recall (P-R) curves across different confidence thresholds, which were logarithmically scaled from 0 to 1.

Defense Experiments

TABLE IV: KF evaluation on defended keyword traffic considers varying settings such as sampling distribution (WTF-PAD), the number of split paths (TrafficSliver), and bandwidth overheads.

Models	BurstGuard	FRONT		WTF-PAD		TrafficSliver	
Settings	-	-	-	normal	normal rcv	path 3	path 5
Bandwidth Overhead (%)	**14**	34	80	38	38	0	0
DKF Accuracy (%)	**42.94**	**63.98**	**61.88**	**49.31**	**49.68**	**17.90**	**10.03**
Tik-Tok Accuracy (%)	15.74	03.07	02.04	07.58	08.23	02.62	02.13
k-FP Accuracy (%)	12.13	10.37	08.06	01.95	09.21	02.46	02.11

We conducted KF for defended keyword traffic with high-performing attack models. Overall, DKF demonstrated robust capabilities in effectively classifying keywords, even against traffic defended by padding with dummy packets. BurstGuard reduced DKF accuracy by 7% more while reducing the bandwidth overhead by 24%.